

Ladder Logic Construction Guide

Micro820 MSG_MODBUS to Automation Direct GS10/GS11

Companion to the *Micro820 MSG_MODBUS Implementation Guide*. This document walks through building every element in Connected Components Workbench (CCW) step by step: creating the project, defining tags, dragging instruction blocks onto rungs, wiring inputs and outputs, configuring the MSG_MODBUS parameters, and chaining messages with .DN bits. Each rung includes a vector-drawn ladder diagram showing the exact structure.

Prerequisite	Complete VFD parameter setup per Implementation Guide
CCW Version	v12.0 or later (v13+ recommended)
Controller	Micro820 (2080-LC20-20QWB or similar)
Language	Ladder Diagram (LD)
Total Rungs	7 rungs (expandable to 10+ with diagnostics)

Contents

Part 1: Create the CCW Project and Add Ladder Program

Part 2: Define All Tags (Variables)

Part 3: Configure the Serial Port

Part 4: Build Rung 0 — Scan Timer (Heartbeat)

Part 5: Build Rung 1 — Write Control Word (MSG_MODBUS)

Part 6: Build Rung 2 — Write Frequency Command (MSG_MODBUS)

Part 7: Build Rung 3 — Read Status Registers (MSG_MODBUS)

Part 8: Build Rung 4 — Map Status to Named Tags

Part 9: Build Rung 5 — Run/Stop Logic with Physical Inputs

Part 10: Build Rung 6 — Error Handling and Retry

Part 11: Download, Go Online, and Test

Appendix: Complete Tag List + Message Chain Diagram

Part 1: Create the CCW Project and Add Ladder Program

1

Open CCW and Create New Project

File > New > Select your Micro820 controller (e.g., 2080-LC20-20QWB).

Name the project **GS10_Modbus_Control**. Click **Create**.

2

Add a Ladder Diagram Program

In Project Organizer, right-click **Programs** > **Add** > **New LD : Ladder Diagram**.

Rename **Prog1** to **ModbusControl**. Double-click to open the ladder editor.

3

Open the Toolbox

Click the **Toolbox** tab (lower-right corner). Expand **Ladder**.

You will drag contacts, coils, timers, and blocks from here onto rungs.

Part 2: Define All Tags (Variables)

Before building any rungs, create all tags in **Global Variables** so they are accessible by HMI and other programs.

1

Open Global Variables

Expand Micro820 controller > double-click **Global Variables**.

2

Create Each Tag from the Table Below

Click the blank row, type **Name**, select **Data Type**, set **Initial Value**.

Tag Name	Data Type	Init	Purpose
ScanTimer	TON	-	Periodic trigger for Modbus cycle
ScanTimerDone	BOOL	FALSE	One-shot pulse from timer .DN
WriteCtrl_MSG	MSG_MODBUS	-	Write Control Word message instance
WriteFreq_MSG	MSG_MODBUS	-	Write Frequency Cmd message instance
ReadStatus_MSG	MSG_MODBUS	-	Read Status message instance
ControlWord	INT	0	Control Word 1 value (write to 0x2000)
FreqCommand	INT	0	Frequency in 0.01 Hz (6000 = 60 Hz)
StatusData	INT[8]	all 0	Array for status read (from 0x2100)
DriveError	INT	0	Mapped from StatusData[0]
DriveStatus	INT	0	Mapped from StatusData[1]
ActualFreq	INT	0	Mapped from StatusData[3]
ActualCurrent	INT	0	Mapped from StatusData[4]
RunCommand	BOOL	FALSE	Operator Run (DI or HMI)
StopCommand	BOOL	FALSE	Operator Stop (DI or HMI)
FwdRevSelect	BOOL	FALSE	FALSE=Fwd, TRUE=Rev
SpeedSetpoint	INT	3000	Speed in 0.01 Hz (3000=30 Hz)
CommOK	BOOL	FALSE	Last read succeeded
CommFault	BOOL	FALSE	MSG .ER fired
RetryCount	INT	0	Consecutive error count

WARNING: MSG_MODBUS instances are created automatically when you drag the instruction block onto a rung (Parts 5-7). You do NOT create them manually in the tag editor.

Part 3: Configure the Serial Port

1

Open Serial Port Config

Click the Micro820 icon > **Serial Port** tab.

2

Set Modbus RTU Master

Driver: Modbus RTU | **Role:** Modbus RTU Master | **Physical Layer:** RS-485

3

Match GS10 Communication Parameters

Baud Rate: 19200 | **Data Bits:** 8 | **Parity:** Even | **Stop Bits:** 1

Must match GS10: P09.01 = 19.2, P09.04 = 14 (8-E-1 RTU)

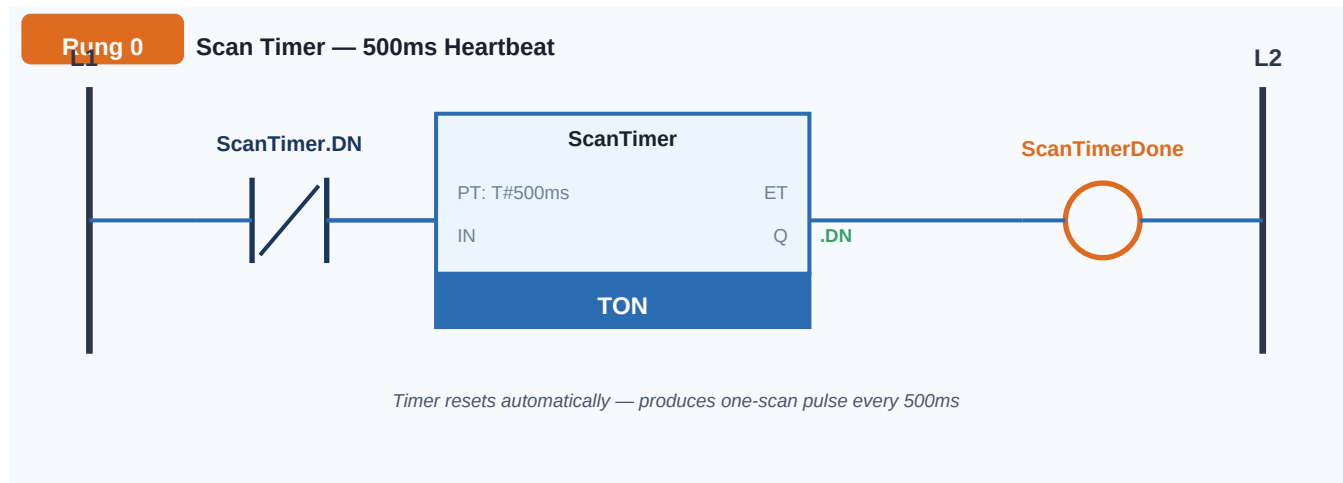
4

Build and Download

Right-click Micro820 > **Build**. Then **Download**. Power-cycle if needed.

Part 4: Build Rung 0 — Scan Timer (Heartbeat)

This rung creates a periodic 500ms trigger pulse that kicks off each Modbus cycle.



How to Build This Rung in CCW

1

Add the TON Timer Block

From **Toolbox > Ladder**, drag **TON** onto the rung.

In the Instance dropdown, create new: **ScanTimer**. Click OK.

2

Set Preset Time

Click the **PT** input. Type **T#500ms**. This is your scan rate.

3

Add Self-Reset Contact (NC)

Drag a **Normally Closed Contact** (---|/|---) to the LEFT of the TON.

Assign it to **ScanTimer.DN**. When DN fires, the NC opens, resetting the timer.

4

Add Output Coil

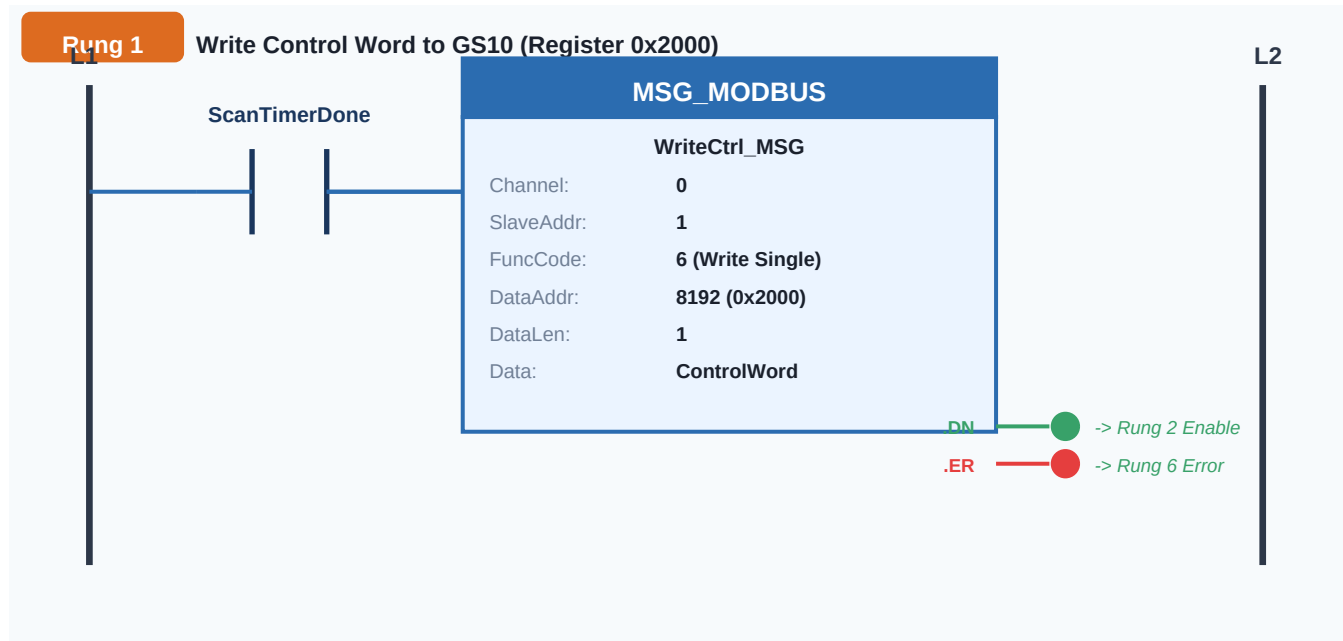
Drag a **Coil** --()-- to the RIGHT. Assign to **ScanTimerDone**.

This BOOL pulses TRUE for one scan each time the timer completes.

TIP: The self-resetting TON is the standard heartbeat pattern in Micro820. Timer counts to 500ms, .DN fires for one scan, NC contact resets the timer, cycle repeats.

Part 5: Build Rung 1 — Write Control Word (MSG_MODBUS)

Writes the Control Word to register 0x2000 (8192) — Run, Stop, Direction, Jog.



How to Build This Rung

1

Add MSG_MODBUS Block

Drag a **Block** onto Rung 1. Type **MSG** in the filter. Select **MSG_MODBUS**.
Create new instance: **WriteCtrl_MSG**.

2

Wire the Enable Input

Drag a **Normally Open Contact** to the LEFT. Assign to **ScanTimerDone**.
This triggers the message when the timer fires.

3

Configure MSG Parameters

Channel: 0 (embedded port) | **SlaveAddr:** 1 (matches P09.00)
FunctionCode: 6 (Write Single Register)
DataAddr: 8192 (0x2000 = Control Word 1)
DataLen: 1 | **Data:** ControlWord (your INT tag)

4

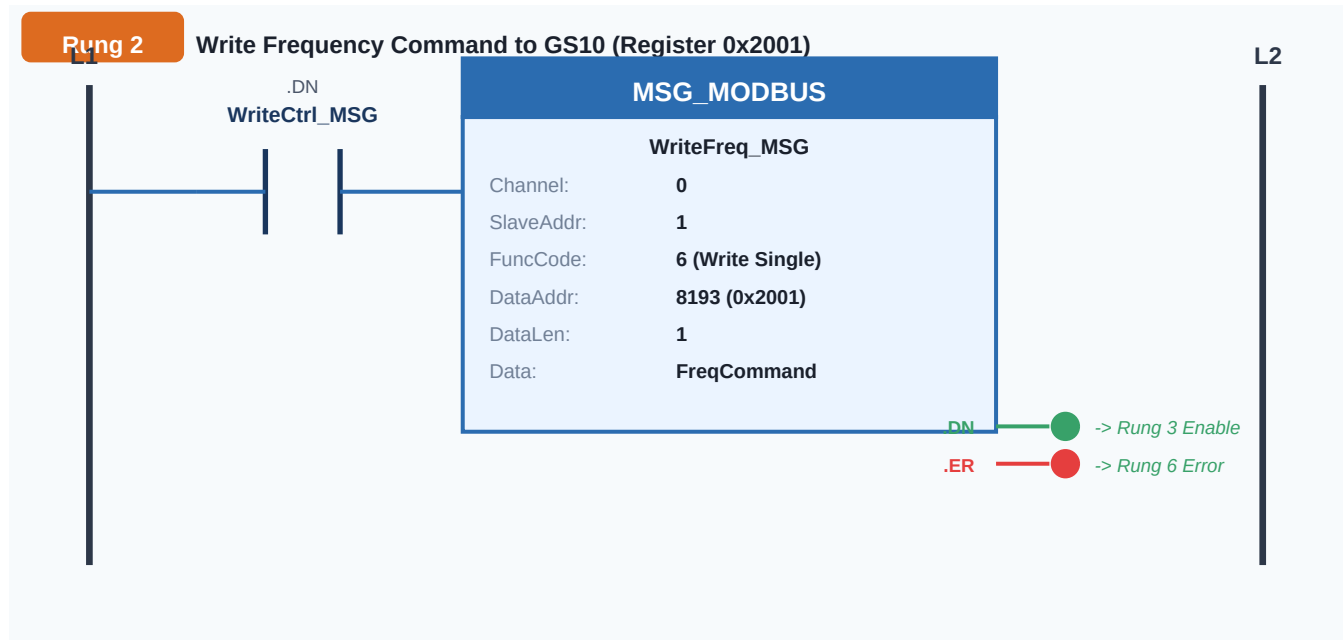
Understand the Outputs

.DN: TRUE when message completes — chains to Rung 2.

.ER: TRUE on failure — chains to Rung 6 error handler.

Part 6: Build Rung 2 — Write Frequency Command

Writes the speed reference to register 0x2001 (8193). Chained from Rung 1.



How to Build This Rung

1

Add MSG_MODBUS

Drag Block, select MSG_MODBUS, create instance: **WriteFreq_MSG**.

2

Chain from Rung 1

Add NO Contact, assign to **WriteCtrl_MSG.DN**. Fires only after Rung 1 completes.

3

Configure

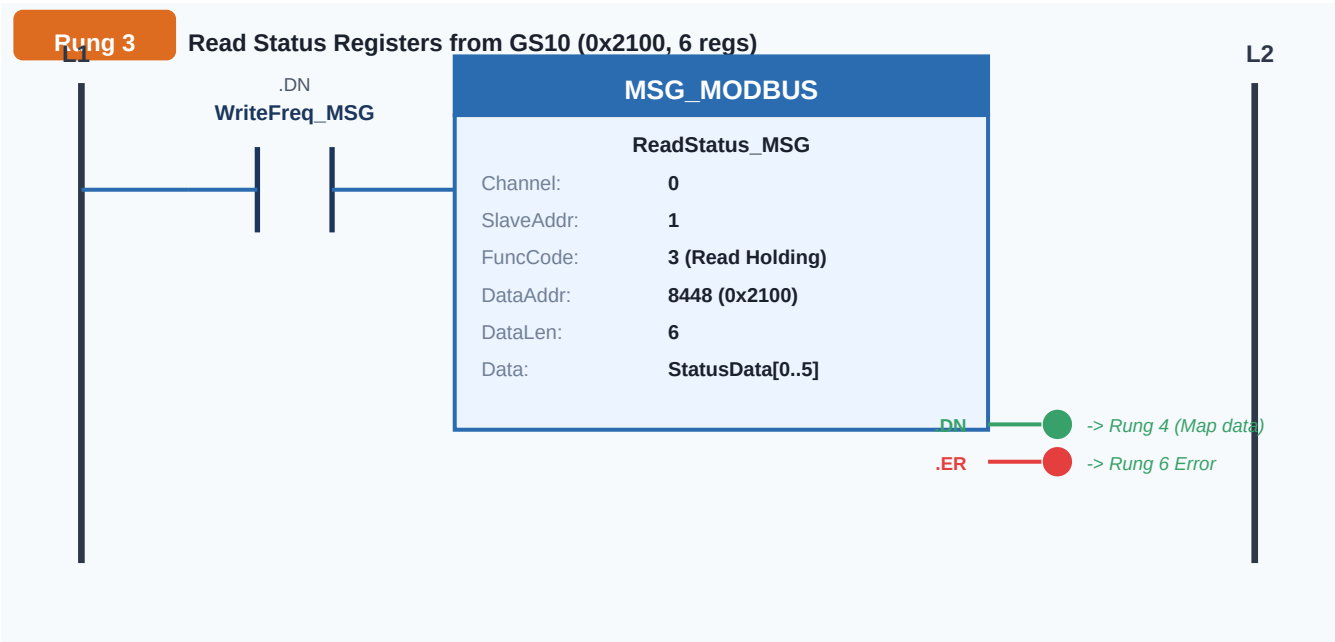
FuncCode: 6 | **DataAddr:** 8193 (0x2001) | **Data:** FreqCommand

Value is 0.01 Hz units: 6000 = 60.00 Hz, 3000 = 30.00 Hz.

TIP: Optimization: Use FC 16 (Write Multiple) to send both ControlWord and FreqCommand in one message. DataAddr=8192, DataLen=2, Data=INT[2] array. Halves your write messages.

Part 7: Build Rung 3 — Read Status Registers

Reads 6 consecutive status registers starting at 0x2100 (8448). Chained from Rung 2.



How to Build + What You Get Back

1

Add MSG_MODBUS

Instance: **ReadStatus_MSG**. Enable: **WriteFreq_MSG.DN**.

2

Configure for Read

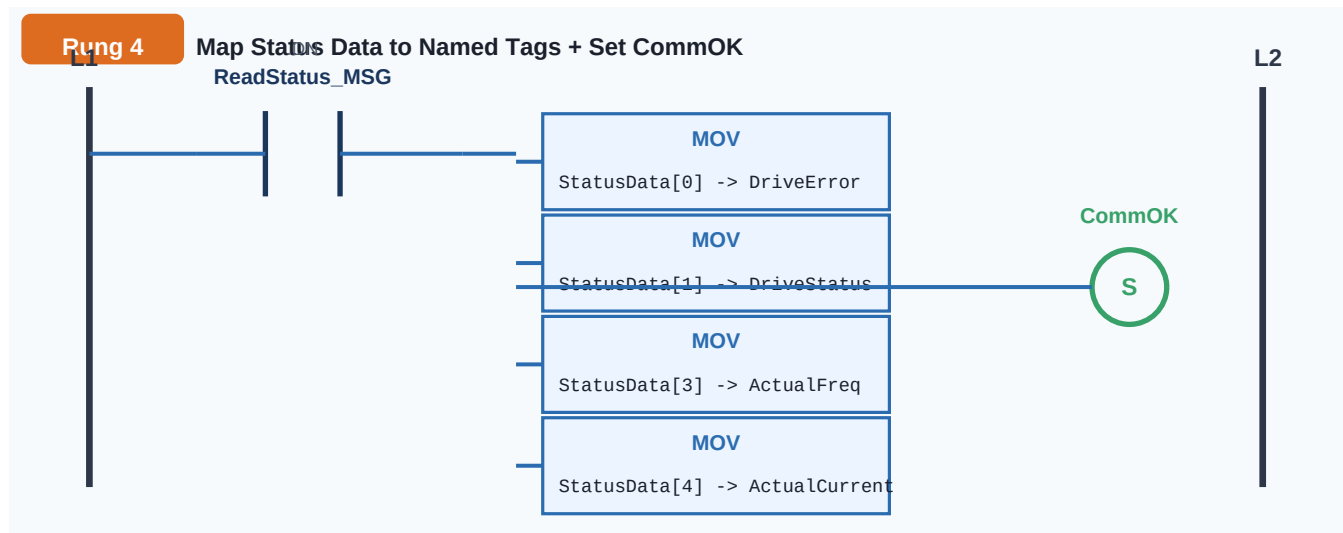
FuncCode: 3 (Read Holding Registers)

DataAddr: 8448 (0x2100) | **DataLen:** 6 | **Data:** StatusData (INT[8])

Index	Register	Content	Units
[0]	0x2100	Error/Fault code	0 = no fault
[1]	0x2101	Drive status bits	run/stop/dir
[2]	0x2102	Freq command	0.01 Hz
[3]	0x2103	Output frequency	0.01 Hz
[4]	0x2104	Output current	0.01 A
[5]	0x2105	DC bus voltage	0.1 V

Part 8: Build Rung 4 — Map Status to Named Tags

Copy array elements to named tags for clarity. Triggered by ReadStatus_MSG.DN.



1

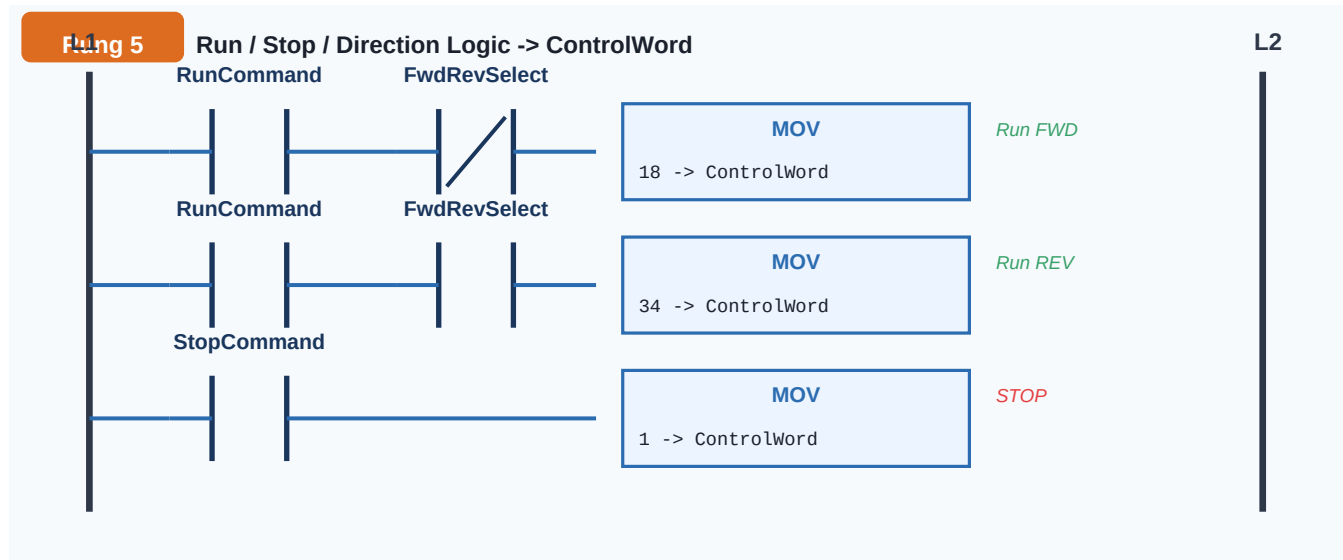
Add MOV Instructions

Drag **MOV** blocks. Source: StatusData[0] > Dest: DriveError. Repeat for each.

Add **SET coil** for CommOK, **RST coil** for CommFault, **MOV 0** to RetryCount.

Part 9: Build Rung 5 — Run/Stop/Direction Logic

Translates operator inputs into the ControlWord integer value.



1

Run Forward Branch

NO contact **RunCommand** + NC contact **FwdRevSelect** -> MOV **18** to ControlWord.
18 = bits 1:0 = 10 (Run) + bits 5:4 = 01 (Forward).

2

Run Reverse Branch

NO **RunCommand** + NO **FwdRevSelect** -> MOV **34** to ControlWord.

3

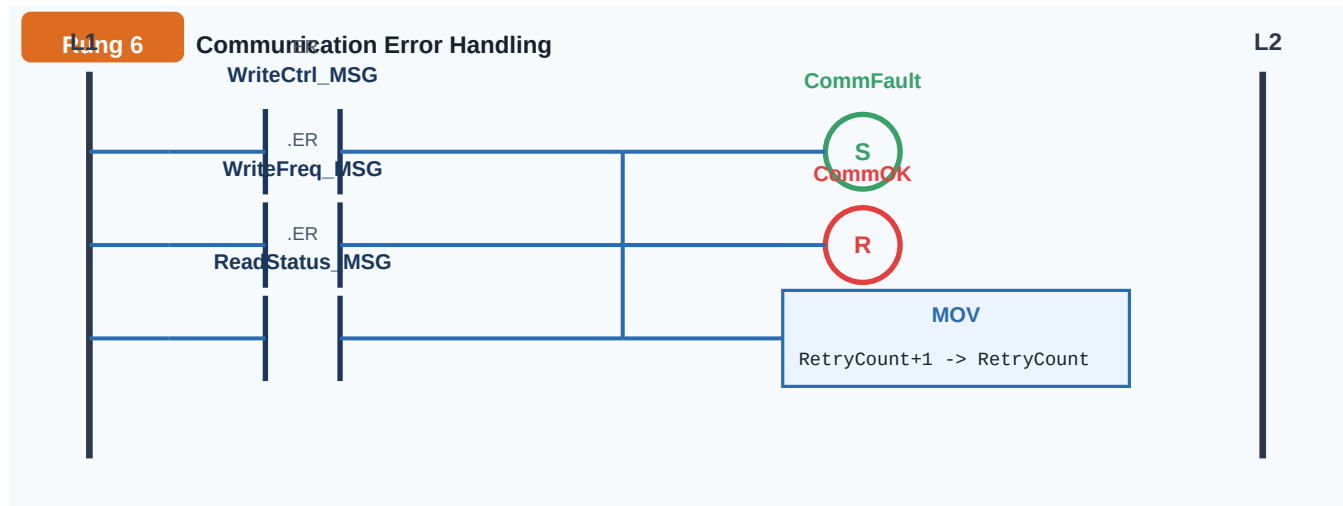
Stop Branch (Priority)

NO **StopCommand** -> MOV **1** to ControlWord. Evaluated last to override run.

A simple unconditional MOV copies SpeedSetpoint to FreqCommand on every scan.

Part 10: Build Rung 6 — Error Handling and Retry

Any MSG .ER bit fires this rung to set CommFault and increment RetryCount.



1

OR the Error Bits

Three parallel NO contacts for WriteCtrl_MSG.ER, WriteFreq_MSG.ER, ReadStatus_MSG.ER.

Any one error triggers the fault outputs.

2

Fault Outputs

SET CommFault | RST CommOK | ADD RetryCount + 1.

After 10+ retries, consider forcing ControlWord = 1 (Stop) for safety.

WARNING: Persistent CommFault = wiring or config problem, not transient. Do not just retry forever. The drive's own P09.02/P09.03 timeout will also catch loss of comms.

Part 11: Download, Go Online, and Test

1

Build

Right-click Micro820 > **Build**. Fix any errors. 0 errors required.

2

Download

Connect via USB or Ethernet. Click **Download**.

3

Go Online and Monitor

Click **Go Online**. Verify ScanTimer is running (ET counting).

Watch MSG .DN bits pulse in sequence: WriteCtrl -> WriteFreq -> ReadStatus.

4

Verify Comms (No Motor)

ControlWord = 0, FreqCommand = 0. Confirm CommOK = TRUE, CommFault = FALSE.

StatusData should populate. DriveError = 0 means no faults.

5

First Motor Test — DISCONNECT LOAD

SpeedSetpoint = 500 (5 Hz), RunCommand = TRUE. Motor should spin slowly.

StopCommand = TRUE -> motor stops. If this works, Modbus chain is operational.

TIP: Use the CCW **Watch Window** (View > Watch Window) to monitor all tags in one place. Add ControlWord, FreqCommand, StatusData[0..5], CommOK, CommFault, and MSG .DN/.ER bits.

Appendix: Message Chain Overview



Control Word Quick Reference

Action	ControlWord	Binary	Notes
No Command	0	0000 0000 0000 0000	Drive maintains current state
Stop	1	0000 0000 0000 0001	Decelerates to stop
Run Forward	18	0000 0000 0001 0010	Run + Forward
Run Reverse	34	0000 0000 0010 0010	Run + Reverse
Jog Forward	19	0000 0000 0001 0011	Jog + Forward
Jog Reverse	35	0000 0000 0010 0011	Jog + Reverse